NEXTGEN PYTHON INTERNSHIP MX · Q2 2026 · NOSQL WORKSHOP

Workshop NoSQL DB
Cassandra

Case study: how Discord stores trillions of chat messages

Follow-along session **Hands-on, all levels welcome**

github.com/agent3dev/workshop-nosql-stage2-cassandra

TWO FLAVORS OF NOSQL

Documents got you here. Distribution gets you further.

MONGODB (PREVIOUS WORKSHOP)

- Flexible **documents**, varying shape
- Runs great on a single node
- Relationships: **embed** or **reference**
- Query however you like, mostly

CASSANDRA (THIS WORKSHOP)

- **Distributed by design** — built for many nodes
- Every table is designed around **one query**
- Data lives where its **partition key** says it does
- You choose consistency vs. availability, per query

Narrow by design, so it can be fast at any scale.

Fun fact: Discord actually lived this exact journey — MongoDB in 2015, outgrown around 100 million messages, migrated to Cassandra in 2017.

THE REAL PROBLEM · DISCORD

Storing trillions of chat messages

- Discord's core read: **"last 50 messages in this channel, newest first"** — needed to be fast, always, at massive scale
- Their real production table partitioned by channel alone — every message in one channel, forever, in ONE partition
- Their largest channels eventually grew that partition big enough to cause **cluster-wide latency** — a real postmortem, not a hypothetical

> *“Lots of concurrent reads as users interact with servers can hotspot a partition... when Discord encountered a hot partition, it frequently affected latency across the entire database cluster.”*

— Discord Engineering: discord.com/blog/how-discord-stores-trillions-of-messages

TODAY, WE REBUILD IT

Curriculum roadmap

#	FILE	SQL PRINCIPLE	CASSANDRA PRINCIPLE
01	`01_crud.cql`	The engine adapts to whatever question you ask	You must know your question before you design the table
02	`02_data_modeling.cql`	Model the **entities**; normalize to avoid duplication	Model the **queries**; duplicate data freely on write
03	`03_partitions_clustering.cql`	Physical storage is the engine's problem, not yours	Where data physically lives is a design decision you own
04	`04_cql_consistency.cql`	One machine, one truth — consistency by default	Many machines — you choose consistency vs. availability

BEYOND TODAY'S SESSION

Pitching Cassandra to a real project

- Not "better modeling" — a narrower bet: you have a problem SQL can't solve on one machine, and you already know your access patterns

✓ **Write volume** that outgrows a single Postgres instance — thousands of writes/sec, sustained

✓ **Data volume** bigger than one server can hold, growing continuously (logs, events, messages)

✓ **Multi-datacenter** — no single point of failure, survives losing an entire data center

✓ **Known, stable access patterns** — not ad-hoc analyst queries

✓ **Tolerance for eventual consistency** on at least some of the data

Fewer than 2–3 of these? Plain SQL is still the right call.

BEYOND TODAY'S SESSION

Who else runs on this exact pattern

- **Discord** — trillions of chat messages, partitioned by channel & time bucket (today's whole case study)
- **IoT / sensor telemetry** — device readings partitioned by `device_id + day`, clustered by timestamp. Same exact shape as today's `messages_by_channel_bucketed` table, just device readings instead of chat messages.
- **Netflix** — viewing history & billing: hundreds of Cassandra clusters, tens of thousands of nodes, petabytes of data, millions of ops/sec
- **Apple** — 75,000+ Cassandra nodes, 10+ petabytes of data, one single cluster over 1,000 nodes

Sources: Netflix Technology Blog; Apple's public Cassandra Summit disclosures (planetcassandra.org, cassandra.apache.org case studies)

COMMON MISCONCEPTIONS

What Cassandra is NOT

- **Not a SQL view.** A view computes a query live over one source of truth — nothing is duplicated. Cassandra's "one table per query" is closer to a **materialized view**: the data is physically stored, shaped for that query. The difference that matters — a materialized view is kept in sync by the database engine automatically; here, **your application** writes to every relevant table itself, on every insert. Nobody does that for you.
- **Not ACID.** A single write to one row (one partition) is atomic and durable — that part is solid. But there are no transactions across multiple partitions or tables, no foreign keys, no rollback spanning several writes. And the "C" in `ONE / QUORUM / ALL` means *replica* consistency — how many copies agree — not the ACID "C" of enforced data invariants.

BEYOND TODAY'S SESSION

Surviving an entire datacenter failure, in practice

- `NetworkTopologyStrategy` replicates **per datacenter** — e.g. `{dc1: 3, dc2: 3}`. Every write goes to both DCs.
- Your app reads/writes at `LOCAL_QUORUM` — quorum **within your own DC only**. It never waits on the far DC.
- If DC2 disappears entirely, clients talking to DC1 at `LOCAL_QUORUM` don't even notice.
- The level to avoid here is `EACH_QUORUM` — it requires every DC to answer, so a dead DC breaks it outright.
- When DC2 comes back: **hinted handoff** replays what it missed (short outages), or `nodetool repair` resyncs it (long outages).

Same mechanism as Exercise 4's node-kill demo — just imagine the nodes you kill all live in the same building.

BEYOND TODAY'S SESSION

How Cassandra compares

SYSTEM	COORDINATION	CONSISTENCY	BEST FOR
`Cassandra`	No leader — gossip + peer nodes	Tunable per query	Fast known-key lookups at massive write scale
`DynamoDB`	No leader — same lineage as Cassandra	Tunable, fully managed by AWS	Same model, zero operations
`CockroachDB / Spanner`	Leader per range (Raft / Paxos)	Strong (serializable) by default	Cross-node JOINs & real transactions
`Elasticsearch`	Elected master (cluster state)	Primary-then-replica, per shard	Full-text search & aggregations, not OLTP lookups

FILE 1 OF 4 · 01_CRUD.CQL

CRUD basics: two design philosophies

SQL (PREVIOUS WORKSHOP)

- The engine adapts to **whatever question** you ask
- Schema comes first; queries adapt to it later, freely

CASSANDRA (TODAY)

- You must know **your query** before you design the table
- No JOIN, no ad-hoc filtering — the table IS the query

CQL

```
SELECT * FROM channels
WHERE server_id = aaaa...;
-- FAILS: server_id isn't the partition key
-- needs ALLOW FILTERING — and even then, it's a full scan
```

- This "refusal" isn't a limitation to work around — it's the setup for file 2's real lesson

FILE 2 OF 4 · 02_DATA_MODELING.CQL

Data modeling: one table per query

- Naive table keyed by `message_id` alone — "messages in this channel" needs a full cluster scan
- Redesign: partition by channel, cluster by time, newest first
- A different query = a different table — duplicating data on write is normal here, not a bug
- This is Discord's actual schema, not a simplification

CQL

```cql
-- naive: no query is fast
PRIMARY KEY (message_id)

-- query-first: Discord's real schema
PRIMARY KEY
  (channel_id, message_id)
-- partition=channel, cluster=time
```

FILE 3 OF 4 · 03_PARTITIONS_CLUSTERING.CQL

Partitions & the hot-partition problem

- Load thousands of messages into ONE channel — `nodetool tablestats` shows a single, huge partition
- The real fix: add a time `bucket` to the partition key
- Same data, 10 smaller partitions instead of 1 — visible live in the stats, not just claimed
- The cost: reads now need to know which bucket(s) to check

CQL

```
-- one giant partition per channel
PRIMARY KEY
  (channel_id, message_id)

-- bucketed: caps partition size
PRIMARY KEY (
  (channel_id, bucket),
  message_id
)
  ↑ the extra ( ) is the fix
```

FILE 4 OF 4 · 04_CQL_CONSISTENCY.CQL

CQL consistency levels

- Real 3-node cluster, replication factor 3 — not a simulation
- `CONSISTENCY ONE / QUORUM / ALL` — how many replicas must answer
- We kill a node live: **ALL** breaks immediately, **QUORUM** and **ONE** keep working
- The tradeoff Discord makes every day: availability and speed over perfect consistency

CQLSH

```
CONSISTENCY QUORUM;
SELECT * FROM users WHERE ...;
-- OK, 2 of 3 replicas answer

-- (kill one node)
CONSISTENCY ALL;
SELECT * FROM users WHERE ...;
-- Cannot achieve consistency ALL
```

BEFORE WE START

Get the cluster running

- `git clone github.com/agent3dev/workshop-nosql-stage2-cassandra`
- `docker compose up -d` — a real 3-node cluster, takes a few minutes to boot
- Check it's ready: `docker exec discord_cassandra1 nodetool status`
- Connect: `docker exec -it discord_cassandra1 cqlsh`

Full setup steps are in the repo README